

南開大學

实验报告

实验 01：利用流式套接字编写聊天程序

姓名：尚文轩

学号：2412456

专业：计算机科学与技术

任课教师：徐敬东、张建忠

2026 年 10 月 21 日

目录

1 实验目的与任务对应 **2**

2 实验环境与工程组织 **2**

3 聊天协议设计 **3**

3.1 协议分层与分帧 3

3.2 编码、文本约束与时间 4

3.3 消息类型与语义 4

3.4 状态、异常与连接关闭 5

3.5 典型交互时序 6

4 程序设计与实现 **6**

4.1 模块与核心函数 6

4.2 整体工作流程 7

4.3 TCP 分帧的核心代码 7

4.4 线程同步和资源生命周期 8

5 实验步骤与测试结果 **8**

5.1 构建与基本操作 8

5.2 功能、边界与异常测试 8

5.3 真实客户端输出 9

5.4 多人并发完整性结果 10

6 数据丢失与 TCP 观测 **10**

6.1 按连接统计的观测方法 10

6.2 抓包方法与证据范围 11

7 遇到的问题与解决方法 **11**

8 结论与局限 **12**

参考文献 **13**

1 实验目的与任务对应

本实验在 Windows 上使用 C++17 和基础 Winsock 函数实现 TCP 聊天室。服务器支持多个客户端同时连接、群聊、一对一私聊、在线列表与上下线通知；客户端采用命令行界面，支持中文昵称、中英文正文和正常退出。所有服务器消息附带本机时钟时间。

实验的重点是应用层协议语法、语义与时序的完整性，以及 TCP 字节流上的正确分帧、多线程共享状态保护和资源回收。实现不使用 CSocket、Qt 网络库、Boost.Asio 或其他现成网络框架。

表 1: 实验要求与实现、证据的对应关系

实验要求	实现位置	验证方式
完整聊天协议	第3节、PROTOCOL.md	首部、类型、状态、异常路径逐项说明
基础 Socket 函数	common.hpp、server.cpp、client.cpp	`socket/bind/listen/accept/connect/send/recv`
界面与正常退出	客户端帮助、/quit	真实客户端进程测试、停机唤醒测试
中文和英文	UTF-8 线协议、UTF-16 控制台输入转换	中文昵称、中文和英文正文往返
多线程与多人聊天	每连接接收线程，房间互斥锁	4 客户端并发，64 会话容量测试
结构清晰、可读	公共协议函数、服务器、客户端分离	模块表、核心函数与流程图
丢失观察、完整交付	TCP 统计、自动化测试、抓包材料	原始日志、可执行文件、报告源码/PDF

2 实验环境与工程组织

实测操作系统为 `Windows11(10.0.26200)`，编译器为 Visual Studio 2019 Build Tools 提供的 MSVC x64；编译参数包含 `/std:c++17/W4/utf-8/O2/MT`，采用静态 C/C++ 运行库，链接 `ws2_32.lib`。报告使用 XeLaTeX 编译，并沿用本人提供模板的南开大学校名图、封面、页眉与目录结构。Python 仅作为测试驱动，不属于聊天实现。

目录或文件	用途
src/common.hpp	线协议、收发循环、编码校验、控制台输入、TCP 观测
src/server.cpp	监听、会话生命周期、用户表、消息分发、停机
src/client.cpp	登录、输入命令解析、接收显示、退出
bin/chat_server.exe	Windows x64 服务器可执行文件
bin/chat_client.exe	Windows x64 客户端可执行文件
tests/	黑盒集成测试与抓包复现脚本
evidence/	真实程序输出、测试结果与观测记录
report/	LaTeX 源码、模板资源、编译后的 PDF
build.ps1、CMakeLists.txt	直接编译和 CMake 构建入口

3 聊天协议设计

3.1 协议分层与分帧

协议名为 CN Chat/1，使用 IPv4 TCP，默认端口为 9001。默认服务器监听 `127.0.0.1`；跨计算机时显式指定服务器实际局域网地址。TCP 提供有序字节流，但不保留一次 `send` 的消息边界，故使用“固定首部 + 长度指定正文”。首部是 16 字节，所有多字节整数采用网络字节序。

表 2: 16 字节首部，按字节偏移排列

偏移	长度	字段	约束
0	4	magic	`0x434E4554`，即 CNET
4	2	version	固定为 1
6	2	type	消息类型，取值 1–12
8	4	payload length	UTF-8 正文长度，最大 16384 字节
12	4	sequence	请求序号或服务器发送标识

首部通过四个 32 位整数显式序列化，版本和类型分别占第二个整数的高、低 16 位；不直接发送带有平台填充的 C++ 结构体。接收时首先完整读取首部，校验 magic、版本、类型和长度，然后才分配正文缓冲并读取正文。发送也必须循环，处理 `send` 只发送部分字节的情况。若 `recv` 返回零或错误而帧未完成，则丢弃该不完整帧，结束会话，不将半条消息广播。

3.2 编码、文本约束与时间

线协议的正文为严格 UTF-8，不允许 NUL 和 CR。业务文本使用 LF 作为字段分隔，正文内容本身不允许 LF。中文昵称长度按 UTF-8 字节计算：昵称为 1–32 字节，不能包含空格、ASCII 控制字符或斜杠；聊天正文为 1–4096 字节。16384 是通用帧长度上限，4096 是聊天正文的业务上限，两者用途不同。

Windows 命令行参数由 `wmain` 接收 UTF-16，控制台读取宽字符，再转换为 UTF-8；输出代码页设置为 UTF-8。重定向管道输入也使用 UTF-8，避免中文在不同系统代码页间隐式转换。

所有服务器业务响应首字段为服务器本机时间，格式为 `YYYY-MM-DDHH:MM:SS`。这是服务器本地时区，不是客户端发送时间，也不用于全局时钟同步。

3.3 消息类型与语义

下表中 T 为服务器时间，N 为昵称，M 为正文，D 为目标昵称；符号 $\backslash n$ 表示单个 LF 字节，而不是反斜杠与字母两个字符。

表 3: 消息语法与处理语义

值	名称	方向	正文与语义
1	HELLO	C → S	N；TCP 建连后首条消息，用于登记昵称
2	WELCOME	S → C	$T\backslash n$ 欢迎说明；确认登录成功
3	CHAT	C → S	M；请求向所有在线用户发送消息
4	MESSAGE	S → C	$T\backslash nN\backslash nM$；群聊投递，包含发送者自己
5	PRIVATE	C → S	$D\backslash nM$；请求私聊
6	DIRECT	S → C	$T\backslash nN\backslash nD\backslash nM$；仅投递发送方与接收方，自聊仅投递一次
7	LIST	C → S	空；请求在线用户列表
8	USERS	S → C	$T\backslash n$ 昵称列表，昵称之间用空格分隔
9	QUIT	C → S	空；请求正常退出
10	BYE	S → C	$T\backslash n$ 退出原因；客户端停止会话
11	NOTICE	S → C	$T\backslash n$ 通知内容；通知其他用户上线或离线
12	ERROR	S → C	$T\backslash n$ 错误说明；可恢复错误保留连接，协议错误关闭连接

每个客户端的请求序号从 1 开始严格递增，包括 HELLO 和 QUIT。服务器维护全局发送标识，每向一个接收者发送一帧增加一次；所以同一群聊在不同客户端上的标识可以不同。私

聊和欢迎消息只发给部分用户，序号间隙不代表消息丢失。标识为 32 位无符号数，理论上按模 2^{32} 回绕；本实验运行量远小于此范围。

3.4 状态、异常与连接关闭

服务器会话状态分为 WAIT_HELLO、ACTIVE 和 CLOSED。握手最长 5 秒，采用独立期限监视线程；即使对端每隔一段时间发送一个字节，也不能无限延长握手。握手完成后不设置空闲接收期限，允许用户长时间不发言。

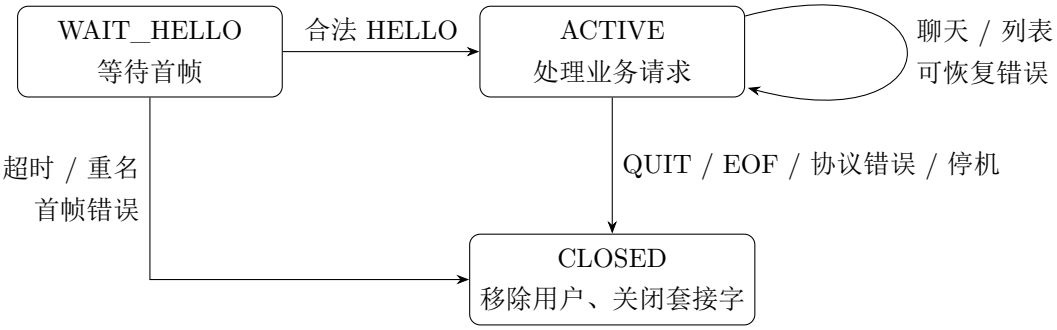


图 1: 会话状态转换图

可恢复错误包括正文为空或过长、私聊格式错误、目标不在线；返回 ERROR 后仍可继续聊天。非法 magic、版本、类型、UTF-8、错误请求序号和状态不允许的类型属于协议错误，返回 ERROR 后关闭。握手超时通过 `shutdown` 中断接收，此时直接 EOF，不能保证再发送 ERROR。

正常关闭时客户端发送 QUIT，服务器返回 BYE，再关闭连接并向其他用户广播离线通知。异常断线时清理用户表并释放昵称。服务器输入 `/quit` 时停止接入，对在线会话发送 BYE，调用 `shutdown` 唤醒阻塞接收，然后逐一等待线程结束，最后清理 Winsock。客户端输入 EOF 等价于退出请求；等待 BYE 最长 3 秒，避免异常对端导致永久等待。

3.5 典型交互时序



图 2: 登录、群聊和退出的正常时序

4 程序设计与实现

4.1 模块与核心函数

模块 / 函数	职责与实现要点
Winsock	RAII 管理 WSAStartup 与 WSACleanup；不是网络框架或套接字封装类
send_all / recv_all	基础 send / recv 循环，推进指针，直到长度满足或连接失败
send_frame / recv_frame	显式字节序转换、16 字节首部、长度检查、UTF-8 验证
name_ok / text_ok	昵称和正文业务校验
serve	单会话工作线程：握手、命令分支、异常清理、统计输出
emit / broadcast	在持有房间锁时发送完整帧；分配全局发送标识
服务器 main	bind/listen、接入线程、停机协调和线程回收
客户端 run / wmain	命令解析与输入发送，接收线程负责异步显示
line	以 100 ms 轮询控制台输入，远端断开时无需再按回车
tcp_stats	WSAIoctl 读取本连接的 TCP 统计；不支持时明确标记 unavailable

4.2 整体工作流程

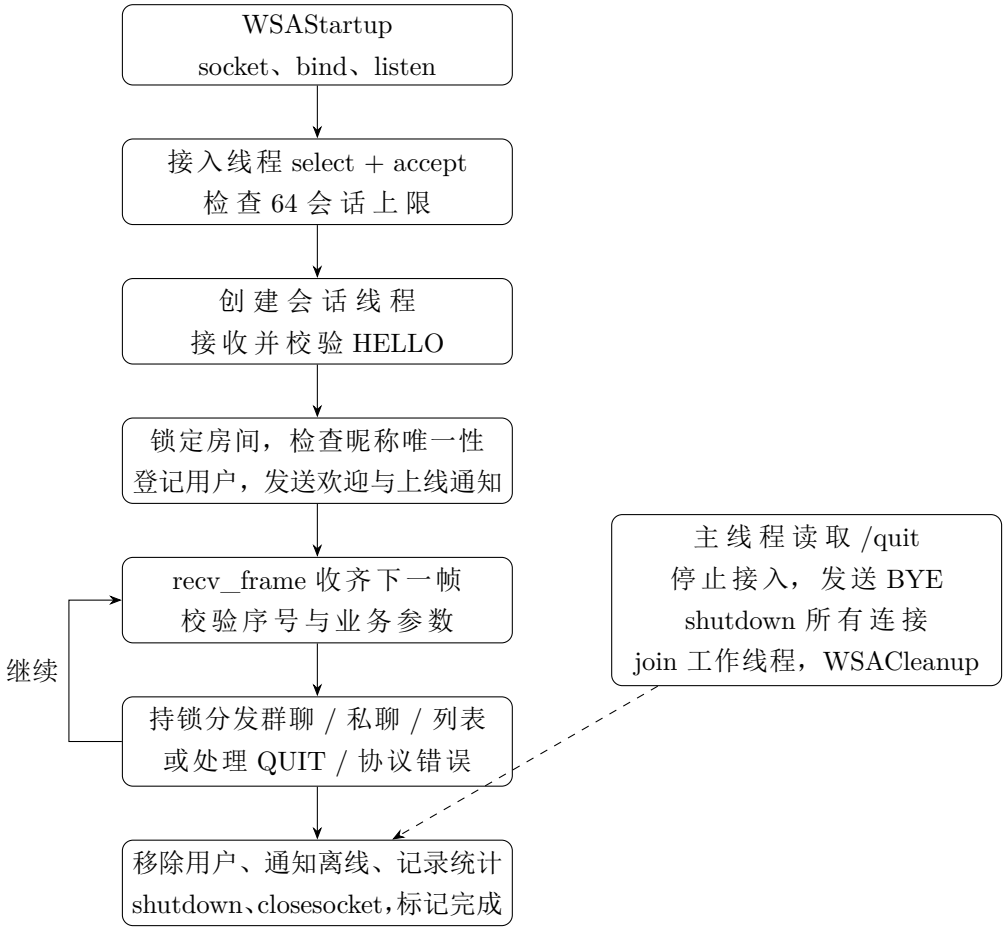


图 3: 服务器整体流程与停机协调

4.3 TCP 分帧的核心代码

```
inline bool recv_all(SOCKET s, char* p, size_t n) {
    while (n) {
        int k = recv(s, p, static_cast<int>(n), 0);
        if (k <= 0) return false;
        p += k;
        n -= k;
    }
    return true;
}
// Validate magic/version/type/length BEFORE resizing the body.
// Read the body with recv_all again, never with one recv call.
```

该循环不会把短读误判为消息结束。长度为零时不发起接收，正好支持 LIST 和 QUIT 的空正文。对应发送循环也不能假定 `send` 一定发送全部数据。网络字节序使用 `htonl/ntohl`，而不是依赖主机端序。

4.4 线程同步和资源生命周期

服务器主线程管理命令输入，接入线程接受连接，每个连接有一个接收线程，握手期间临时增加一个期限监视线程。客户端主线程处理输入，接收线程显示服务器消息。在线用户表、昵称、活跃状态和服务器发送标识受同一房间互斥锁保护。接收与等待用户输入时不占用房间锁；只有状态变更和分发发送时持锁。该简单模型确保多个工作线程不会在同一客户端 TCP 字节流中交叉写入帧，也保证各接收者看到一致的群聊次序。

```
std::lock_guard<std::mutex> lock(room);
// Validate request sequence and dispatch under the same lock.
if (f.type == Chat) {
    // First validate the UTF-8 text and its byte length.
    broadcast(Message, s->name + "\n" + f.body);
}
```

发送设置 2 秒超时，单个慢接收者不能永久阻塞房间；但全局锁内发送仍可能在多个慢连接下累积等待，不适合大规模生产服务。并发会话上限为 64；已完成线程在下一次接入时回收，服务器停机时回收剩余线程。会话工作线程负责关闭自身套接字，协调线程仅调用 shutdown，避免重复关闭和句柄复用错误。

5 实验步骤与测试结果

5.1 构建与基本操作

在工程根目录的 PowerShell 中执行：

```
.\lab01\build.ps1
.\lab01\bin\chat_server.exe 127.0.0.1 9001
```

随后分别打开三个终端运行客户端（昵称可替换为中文）：

```
.\lab01\bin\chat_client.exe 127.0.0.1 9001 Alice
.\lab01\bin\chat_client.exe 127.0.0.1 9001 Bob
.\lab01\bin\chat_client.exe 127.0.0.1 9001 Carol
```

直接输入文字为群聊；`/msgBobhello` 为私聊；`/list` 查看在线用户；`/help` 查看提示；`/quit` 正常退出。服务端也通过 `/quit` 停机。跨主机使用服务器实际局域网 IPv4 地址，并由操作者按实际网络策略允许该端口；本次自动化验证的地址均为回环地址。

5.2 功能、边界与异常测试

执行 `pythonlab01/tests/integration.py` 启动真实 C++ 可执行程序并驱动协议请求，结果写入 evidence 目录。测试不是替身服务，也没有绕开服务器业务实现。

表 4: 黑盒集成测试覆盖范围

编号	操作及验证目标	结果
01	三用户中文昵称、中英文及表情群聊、在线列表	通过
02	私聊双方可见、第三方无消息、目标不在线	通过
03	首帧逐字节发送，三帧合并发送	通过
04	拒绝重复昵称，退出后可复用	通过
05	非法 magic、版本、类型、过大长度、UTF-8	通过
06	错误序号、客户端发送服务器专用类型	通过
07	空正文、4097 字节拒绝，4096 字节通过	通过
08	突然断线后的通知、昵称释放	通过
09	只发送一个首部字节后超时；半帧后 EOF	通过
10	四用户并发，逐条校验完整性、重复与次序	通过
11	真实客户端中文参数、群聊、列表、自聊、退出	通过
12	客户端不输入时服务器关闭，客户端自行退出	通过
13	同一昵称连续连接、退出 80 次	通过
14	64 会话通过，第 65 条连接及时关闭	通过

本次记录共 14 项测试，失败 0 项、错误 0 项，总耗时 18.132 秒。测试时间为 2026-10-21 23:30:48 +0800。完整结果为 `evidence/test-results.txt`，可机器读取的结果为 `evidence/results.json`。

5.3 真实客户端输出

以下为测试 11 中真实客户端进程的 UTF-8 标准输出排版，不是人工拼接的截图。该过程由管道输入驱动；人工交互界面的输入行为仍应在展示前现场试运行。

```
2026-10-21 23:30:42
欢迎 客户端小明
输入文字群聊 | /msg 昵称 内容 私聊 | /list 在线列表 | /help 帮助 | /quit 正常退出
[群聊 #3] 2026-10-21 23:30:42 客户端小明 你好 Hello
[在线 #4] 2026-10-21 23:30:42 客户端小明
[私聊 #5] 2026-10-21 23:30:42 客户端小明 客户端小明 自己的私聊
[系统 #6] 2026-10-21 23:30:42 正常退出
客户端已退出
```

5.4 多人并发完整性结果

四个客户端各发送 300 条唯一编号消息，总计 1200 条。每条广播包含发送者自己，故预期投递总量为

$$N_{\text{expected}} = N_{\text{clients}} \times N_{\text{messages}} = 4 \times 1200 = 4800.$$

测试为每个接收者建立独立读取任务，以“发送者昵称 + 消息编号 + 正文”逐条比较预期集合，同时验证没有重复、所有接收者观察到的广播正文次序相同。

表 5: 回环环境中的应用层完整性实测

指标	实测值
并发客户端数	4
发送消息数	1200
预期投递数	4800
实际收到数	4800
缺失投递数 / 重复投递数	0 / 0
发送至全部收齐耗时	0.109 s

该耗时只反映本次本机自动化测试，不是局域网吞吐率测试，也不应外推为远程链路性能。

6 数据丢失与 TCP 观测

6.1 按连接统计的观测方法

服务器在关闭套接字之前，通过基础 Winsock 的 `WSAIoctl(SIO_TCP_INFO)` 获取 `TCP_INFO_v0`。记录本端发送、接收字节数、重传字节数、重复 ACK 和重传超时事件。字段含义参考 Microsoft 文档 [1]。查询失败时输出 unavailable 及错误码，不将查询失败记为零。

表 6: 并发测试中服务器四条连接的 TCP 统计

连接	发出字节	收到字节	重传字节	重复 ACK	超时事件
User0	73392	10521	0	0	0
User1	73360	10521	0	0	0
User2	73312	10521	0	0	0
User3	73264	10521	0	0	0

本次四条服务器连接均未记录发送方向的重传字节、重复 ACK 或重传超时事件，应用层的 4800 次预期投递也全部完成。因此，可以报告**本次回环测试未观察到应用层消息缺失，服务器发送方向未记录 TCP 重传**。不能将其写成“网络一定没有丢包”：TCP 可能通过重传掩盖

底层丢失；本端统计的发送方向不能代表对端发送方向；回环测试也没有经过真实网线或无线信道。

6.2 抓包方法与证据范围

在用户批准 Windows UAC 提升权限后，使用系统 Pktmon 对 TCP 9001 端口进行独立录制，保存原始 `chat.etl`、原始转换结果 `chat.pcapng`、仅丢弃事件的 `chat-drops.pcapng` 和文本 `chat-packets.txt`。该次负载为 3 个用户、20 条消息、60 次经正文断言验证的投递，全部成功。原始跟踪头的 EventsLost 和 BuffersLost 均为 0，表示跟踪系统未报告事件缓冲丢失。

实际捕获并非“没有丢弃事件”。本次回环抓包点没有记录正常路径数据包，但记录了 2 个 `INET:FIN-WAIT2` 丢弃事件。对保存的两个 IP 数据报逐字节解析后，均为 127.0.0.1 上客户端临时端口发往服务器 9001 的 TCP 段，IP 总长度 40 字节，TCP 标志为 `0x10`（仅 ACK），应用载荷为 0 字节。它们出现在 QUIT/BYE 关闭阶段，所以这两条记录体现的是本机 TCP 状态处理对控制段的丢弃，不能直接等同于聊天正文丢失。该次服务端三个连接的重传字节也均为 0。

默认 `etl2pcap` 不包含 drop 事件，因此其结果只有文件头；必须加 `--drop-only` 才能导出这两个真实数据报。本机导出器的接口描述标为 Ethernet，而记录内容是原始 IPv4；提供的 `chat-drops-raw.pcap` 仅将相同原始字节放入正确的 LINKTYPE_RAW 分析容器，原始 ETL 与 PCAPNG 均保留，便于复核。结构化逐包结果见 `capture-summary.json`，原始丢弃原因节选见 `capture-excerpt.txt`。

观测限制：正常回环流量没有被本抓包点完整记录，因此不能用“2 个丢弃记录 / 全部 TCP 段数”计算丢包率，也不能从空的普通 PCAPNG 推导零丢包。本次结论分别保留“应用消息未缺失”“本端发送未记录重传”与“关闭阶段观察到两个无载荷 ACK 被丢弃”三项事实。

独立复现时，可使用安装 Npcap 的 Wireshark 在回环适配器抓取本机聊天，或在真实网卡抓取跨主机聊天，显示过滤器为 `tcp.port==9001`。分析建连的 SYN/SYN-ACK/ACK、聊天载荷、正常退出的连接关闭，并检查 `tcp.analysis.retransmission`、`tcp.analysis.fast_retransmission`、`tcp.analysis.lost_segment` 和 `tcp.analysis.duplicate_ack`。这些标签是分析器的推断；抓包点本身漏采、乱序或同一包在多个组件重复记录都可能影响判断。Pktmon 的多组件副本不能直接视为 TCP 重传；转换 PCAPNG 时应选定单个组件并按 TCP 序号与确认号核对。

7 遇到的问题与解决方法

1. **旧编译器不支持标准多线程。**默认 MinGW 缺少可用的标准线程实现，改用已安装的 MSVC C++17。提供 MSVC 自动发现脚本和现代 MinGW-w64 的备用入口，不依赖旧 IDE 工程。
2. **TCP 消息可能拆分或合并。**使用 16 字节固定首部与长度字段，循环收齐；以逐字节首帧和合并多帧验证处理结果。
3. **中文昵称受到系统代码页影响。**Windows 参数使用 wmain，显式从 UTF-16 转换到 UTF-8，网络接收时进行严格 UTF-8 校验。

4. **多个线程可能交叉写一条 TCP 流。**将共享状态变更和完整帧发送纳入房间互斥锁，验证四客户端广播正文的观察次序相同。
5. **远端关闭后客户端仍可能等待输入。**控制台输入以 100 ms 轮询运行状态，接收线程在 EOF 或 BYE 后清除运行标志，主线程退出。
6. **握手慢速发送拖延资源回收。**使用总时限监视，而不是每次 recv 重新开始计时；5 秒后 shutdown 唤醒接收。
7. **对超时关闭的错误响应作了错误假设。**初始测试要求超时后收到 ERROR，但 shutdown 已关闭收发，正确可观察结果是 EOF。将断言修正为期限内关闭，并保留对非法完整首部返回 ERROR 的验证。

8 结论与局限

本实验完成了基于基础 Winsock API 的 C++ 多线程 TCP 聊天程序，覆盖中英文、多人群聊、一对一私聊、时间戳、在线列表和正常退出，并通过 14 项黑盒测试。应用层逐条校验结果与按连接 TCP 统计共同支持本次实验的观测结论。

当前采用全局锁串行分发，适用于有限人数的教学实验；面对多个慢接收者会产生累积等待。协议没有加密、账户认证、离线消息或持久化；客户端名称只是在当前房间内唯一的显示标识。自动化测试在本机回环完成，不能替代实验室跨主机环境的现场运行。

参考文献

[1] Microsoft. TCP_INFO_v0 structure (mstcpip.h). https://learn.microsoft.com/en-us/windows/win32/api/mstcpip/ns-mstcpip-tcp_info_v0.

[2] Microsoft. Windows Sockets 2. https://learn.microsoft.com/en-us/windows/win32/winsock/windows-sockets-start-page-2.

[3] Microsoft. Packet Monitor (Pktmon) command syntax. https://learn.microsoft.com/en-us/windows-server/networking/technologies/pktmon/pktmon-syntax.